

KDEllipsy documentation

Alex Villarroel Carrasco 2

May 11, 2026

Contents

1	Theoretical Background	2
1.1	Problem Definition	2
1.2	Forward Modeling Concepts	2
1.3	Inversion Perspective	2
1.4	Assumptions and Limitations	2
2	User Manual	3
2.1	Installation	3
2.2	Project Structure	3
2.3	Input Data	3
2.4	Running an Inversion	3
2.5	Outputs	3
3	Usage Examples	4
3.1	Example 1: Basic Kinematic Inversion	4
3.2	Example 2: Tuning Inversion Parameters	4
3.3	Example 3: Dynamic Legacy Workflow	4
3.4	Interpreting Results	4
4	Developer Notes	5
4.1	Software Architecture	5
4.2	Performance Considerations	5
4.3	Validation Strategy	5
4.4	Roadmap	5

Chapter 1

Theoretical Background

1.1 Problem Definition

KDEllipsy addresses earthquake source inversion by combining physical modeling and waveform fitting. The goal is to estimate source parameters that explain observed seismic records.

1.2 Forward Modeling Concepts

The forward problem maps source parameters to synthetic seismograms. In this framework, source geometry, rupture timing, and source amplitudes are translated into predicted waveforms at receiver stations.

1.3 Inversion Perspective

The inverse problem searches for parameter sets that minimize a misfit between observed and synthetic waveforms. Depending on the workflow, this can be done with stochastic optimization (e.g., Neighbourhood Algorithm) or Bayesian sampling (e.g., MCMC).

1.4 Assumptions and Limitations

This section should document key assumptions, such as fixed mesh geometry, velocity model choices, and data-window selection for misfit computation.

Chapter 2

User Manual

2.1 Installation

Describe installation requirements (Python version, dependencies, and optional tools), and include project setup commands.

2.2 Project Structure

Explain the repository layout, including `kdellipsy/`, `Dynamic_inversion/`, and documentation folders.

2.3 Input Data

Document required input files, expected formats, units, and naming conventions.

2.4 Running an Inversion

Provide the command-line workflow for running kinematic and dynamic workflows, including required flags and output directories.

2.5 Outputs

Describe generated artifacts such as JSON/CSV results, figures, and intermediate files.

Chapter 3

Usage Examples

3.1 Example 1: Basic Kinematic Inversion

Provide a minimal end-to-end run with default parameters and expected outputs.

3.2 Example 2: Tuning Inversion Parameters

Show how to modify search settings (iterations, proposal scales, or sampling choices) and discuss practical effects.

3.3 Example 3: Dynamic Legacy Workflow

Document how to execute the legacy dynamic workflow and where to inspect outputs.

3.4 Interpreting Results

Summarize how to read misfit evolution, best-model parameters, and waveform fit plots.

Chapter 4

Developer Notes

4.1 Software Architecture

Explain the separation between core modules, inversion modules, input parsing, and workflows.

4.2 Performance Considerations

Document known bottlenecks and optimization strategies, such as Green’s function caching and source indexing consistency.

4.3 Validation Strategy

Describe how to validate physics and implementation changes using synthetic tests and reproducible runs.

4.4 Roadmap

List planned features and refactoring goals (for example, dynamic integration and unified inversion interfaces).